

Solved Past Papers

Multiple Linear Regression – Step by Step Solution

Given Data: A school wants to study how **study hours** (x_1) and **sleep hours** (x_2) affect the **exam score** (y).

x_1 (Study Hours)	x_2 (Sleep Hours)	y (Exam Score)
1	6	60
2	7	65
3	8	70

Regression Model

The multiple linear regression model is:

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

where:

- y = predicted exam score
- x_1 = study hours
- x_2 = sleep hours
- θ_0 = intercept
- θ_1, θ_2 = coefficients

Step 1: Matrix Representation

Design Matrix X

$$X = \begin{bmatrix} 1 & 1 & 6 \\ 1 & 2 & 7 \\ 1 & 3 & 8 \end{bmatrix}$$

Output Vector y

$$y = \begin{bmatrix} 60 \\ 65 \\ 70 \end{bmatrix}$$

Step 2: Normal Equation

$$\theta = (X^T X)^{-1} X^T y$$

Step 3: Transpose of X

$$X^T = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 6 & 7 & 8 \end{bmatrix}$$

Step 4: Compute $X^T X$

$$X^T X = \begin{bmatrix} 3 & 6 & 21 \\ 6 & 14 & 46 \\ 21 & 46 & 149 \end{bmatrix}$$

Step 5: Compute $X^T y$

$$X^T y = \begin{bmatrix} 195 \\ 400 \\ 1375 \end{bmatrix}$$

Step 6: Solve the System

$$\begin{cases} 3\theta_0 + 6\theta_1 + 21\theta_2 = 195 \\ 6\theta_0 + 14\theta_1 + 46\theta_2 = 400 \\ 21\theta_0 + 46\theta_1 + 149\theta_2 = 1375 \end{cases}$$

Solving the above equations gives:

$$\theta_0 = 45, \quad \theta_1 = 5, \quad \theta_2 = 0$$

Final Regression Equation

$$\boxed{y = 45 + 5x_1 + 0x_2}$$

Step 7: Prediction

For a student who studies 4 hours and sleeps 9 hours:

$$y = 45 + 5(4) + 0(9)$$

$$y = 45 + 20 = \boxed{65}$$

Step 8: Interpretation

- $\theta_0 = 45$: Base score when study and sleep hours are zero.
- $\theta_1 = 5$: Each extra hour of study increases score by 5 marks.
- $\theta_2 = 0$: Sleep hours have no effect in this dataset.

Final Answer

$$\boxed{\theta_0 = 45, \quad \theta_1 = 5, \quad \theta_2 = 0}$$

$$\boxed{\text{Predicted score for } x_1 = 4, x_2 = 9 \text{ is } 65}$$

KMeans++ - Problem Statement

We apply **K-Means++ clustering** on a 2-D dataset with:

$$k = 2$$

Given Data

Point	x	y
P_1	1	2
P_2	2	3
P_3	3	3
P_4	8	7
P_5	9	8
P_6	10	9

K-Means++ Initialization

Step 1: Choose First Centroid Randomly

Assume the first centroid is selected as:

$$C_1 = (3, 3)$$

Step 2: Compute Squared Distances

Squared Euclidean distance:

$$D^2 = (x - x_c)^2 + (y - y_c)^2$$

Point	D^2 from C_1
P_1	$(1 - 3)^2 + (2 - 3)^2 = 5$
P_2	$(2 - 3)^2 + (3 - 3)^2 = 1$
P_3	$(3 - 3)^2 + (3 - 3)^2 = 0$
P_4	$(8 - 3)^2 + (7 - 3)^2 = 41$
P_5	$(9 - 3)^2 + (8 - 3)^2 = 61$
P_6	$(10 - 3)^2 + (9 - 3)^2 = 85$

Step 3: Compute Probabilities

$$\sum D^2 = 5 + 1 + 0 + 41 + 61 + 85 = 193$$

$$P(i) = \frac{D_i^2}{\sum D^2}$$

Point	D^2	$P(i)$
P_1	5	$5/193 = 0.026$
P_2	1	$1/193 = 0.005$
P_3	0	0
P_4	41	$41/193 = 0.212$
P_5	61	$61/193 = 0.316$
P_6	85	$85/193 = 0.440$

Step 4: Choose Second Centroid

Highest probability corresponds to:

$$C_2 = (10, 9)$$

Initial Centroids

$$C_1 = (3, 3), \quad C_2 = (10, 9)$$

Iteration 1: Assignment Step

Point	$d(C_1)$	$d(C_2)$	Cluster
(1, 2)	$\sqrt{5}$	$\sqrt{145}$	1
(2, 3)	1	$\sqrt{113}$	1
(3, 3)	0	$\sqrt{85}$	1
(8, 7)	$\sqrt{41}$	$\sqrt{8}$	2
(9, 8)	$\sqrt{61}$	$\sqrt{2}$	2
(10, 9)	$\sqrt{85}$	0	2

Clusters After Iteration 1

$$\text{Cluster 1} = \{(1, 2), (2, 3), (3, 3)\}$$

$$\text{Cluster 2} = \{(8, 7), (9, 8), (10, 9)\}$$

Update Centroids

$$C_1 = \left(\frac{1+2+3}{3}, \frac{2+3+3}{3} \right) = (2, 2.67)$$

$$C_2 = \left(\frac{8+9+10}{3}, \frac{7+8+9}{3} \right) = (9, 8)$$

Iteration 2

Cluster assignments do not change. Hence, the algorithm converges.

Final Clusters

$$\text{Cluster A} : \{(1, 2), (2, 3), (3, 3)\}, C_1 = (2, 2.67)$$

$$\text{Cluster B} : \{(8, 7), (9, 8), (10, 9)\}, C_2 = (9, 8)$$

SSE Calculation

Cluster A

$$SSE_1 = 2.78 + 0.11 + 1.11 = 4.00$$

Cluster B

$$SSE_2 = 2 + 0 + 2 = 4.00$$

Total SSE

$$SSE = 8.00$$

Conclusion

K-Means++ selects well-spread initial centroids, leading to faster convergence and lower error.

Q1. Short Questions (Conceptual Understanding)

(a) Difference between Biological and Artificial Neurons

Biological Neurons:

- Exist in the human brain
- Receive signals through dendrites
- Process signals in the cell body
- Transmit signals via axons
- Learning occurs through synaptic strength changes

Artificial Neurons:

- Mathematical models inspired by biological neurons
- Take numerical inputs and weights
- Compute weighted sum and apply activation function
- Learning occurs by updating weights using algorithms

Key Difference: Biological neurons are physical and complex, while artificial neurons are simplified mathematical abstractions.

(b) Why Do We Need a Learning Algorithm Even When True Labels Are Known?

Although true labels are known, manually updating weights is:

- Inefficient for large datasets
- Error-prone
- Not scalable to complex models

A learning algorithm:

- Systematically minimizes error
- Finds optimal weights automatically
- Ensures convergence to a solution

Conclusion: Learning algorithms automate and optimize the weight update process.

(c) Why Is Linear Separability Desired?

Linear separability means that data classes can be separated using a straight line (or hyper-plane).

- Simplifies classification
- Leads to faster training
- Guarantees convergence for algorithms like Perceptron

Note: If data is not linearly separable, nonlinear models or hidden layers are required.

(d) Role of Hidden Layers in a Neural Network

Hidden layers:

- Transform inputs into higher-level representations
- Learn complex patterns and non-linear relationships
- Increase model capacity and expressiveness

Without hidden layers: only linear relationships can be learned.

With hidden layers: complex functions and decision boundaries are possible.

(e) Role of Chain Rule in Gradient Computation

The chain rule allows us to:

- Break complex derivatives into simpler parts
- Compute gradients layer-by-layer
- Efficiently update weights using backpropagation

Mathematically:

$$\frac{dL}{dw} = \frac{dL}{dy} \cdot \frac{dy}{dz} \cdot \frac{dz}{dw}$$

Importance: Without the chain rule, gradient computation would be infeasible for deep networks.

(f) Importance of Conditional Independence in Naive Bayes

Naive Bayes assumes:

$$P(x_1, x_2, \dots, x_n | y) = \prod P(x_i | y)$$

- Simplifies probability computation
- Reduces parameter estimation complexity
- Makes the model computationally efficient

Even if assumption is not fully true, Naive Bayes often performs well in practice.

(g) Choosing Number of Principal Components in PCA

Common methods include:

- **Explained Variance Ratio**: retain components explaining 90–95% variance
- **Scree Plot**: choose elbow point
- **Application Needs**: balance accuracy and dimensionality reduction

Goal: Reduce dimensions while preserving most information.

(h) Linkage Criteria in Hierarchical Clustering

Linkage criteria define how distance between clusters is computed.

- **Single Linkage**: minimum distance (can cause chaining)
- **Complete Linkage**: maximum distance (compact clusters)
- **Average Linkage**: average distance
- **Ward's Method**: minimizes variance within clusters

Effect: Different linkage methods produce different dendrogram structures and cluster shapes.

1 Introduction

We perform Agglomerative Clustering using the **single-linkage** method on the given distance matrix. In single-linkage clustering, the distance between two clusters is defined as the ****minimum distance between any pair of objects**** from each cluster.

2 Step 0: Initial Distance Table

	<i>RO1</i>	<i>RO2</i>	<i>RO3</i>	<i>RO4</i>	<i>RO5</i>	<i>RO6</i>	<i>RO7</i>
<i>RO1</i>	0	936	417	681	515	765	849
<i>RO2</i>	936	0	559	344	255	690	702
<i>RO3</i>	417	559	0	702	640	600	780
<i>RO4</i>	681	344	702	0	995	725	930
<i>RO5</i>	515	255	640	995	0	559	570
<i>RO6</i>	765	690	600	725	559	0	702
<i>RO7</i>	849	702	780	930	570	702	0

3 Step 1: Merge Closest Offices

- ****Find minimum distance****: 255 (between *RO2* and *RO5*). - Merge them into cluster $C1 = \{RO2, RO5\}$. - ****Update distances****: Use single-linkage, i.e., distance between a cluster and a point = minimum distance between any member of the cluster and that point.

$$d(C1, RO1) = \min(d(RO2, RO1), d(RO5, RO1)) = \min(936, 515) = 515$$

$$d(C1, RO3) = \min(d(RO2, RO3), d(RO5, RO3)) = \min(559, 640) = 559$$

$$d(C1, RO4) = \min(d(RO2, RO4), d(RO5, RO4)) = \min(344, 995) = 344$$

$$d(C1, RO6) = \min(d(RO2, RO6), d(RO5, RO6)) = \min(690, 559) = 559$$

$$d(C1, RO7) = \min(d(RO2, RO7), d(RO5, RO7)) = \min(702, 570) = 570$$

Updated distance table:

	<i>RO1</i>	<i>C1</i>	<i>RO3</i>	<i>RO4</i>	<i>RO6</i>	<i>RO7</i>
<i>RO1</i>	0	515	417	681	765	849
<i>C1</i>	515	0	559	344	559	570
<i>RO3</i>	417	559	0	702	600	780
<i>RO4</i>	681	344	702	0	725	930
<i>RO6</i>	765	559	600	725	0	702
<i>RO7</i>	849	570	780	930	702	0

4 Step 2: Merge Next Closest Offices

- ****Minimum distance****: 344 (between *RO4* and *C1*). - Merge into $C2 = \{RO2, RO5, RO4\}$.
- ****Update distances****

$$d(C2, RO1) = \min(d(RO4, RO1), d(C1, RO1)) = \min(681, 515) = 515$$

$$d(C2, RO3) = \min(d(RO4, RO3), d(C1, RO3)) = \min(702, 559) = 559$$

$$d(C2, RO6) = \min(d(RO4, RO6), d(C1, RO6)) = \min(725, 559) = 559$$

$$d(C2, RO7) = \min(d(RO4, RO7), d(C1, RO7)) = \min(930, 570) = 570$$

Updated table:

	<i>RO1</i>	<i>C2</i>	<i>RO3</i>	<i>RO6</i>	<i>RO7</i>
<i>RO1</i>	0	515	417	765	849
<i>C2</i>	515	0	559	559	570
<i>RO3</i>	417	559	0	600	780
<i>RO6</i>	765	559	600	0	702
<i>RO7</i>	849	570	780	702	0

5 Step 3: Merge Next Closest Offices

- **Minimum distance**: 417 (between *RO1* and *RO3*). - Merge into $C3 = \{RO1, RO3\}$. -
 Update distances:

$$d(C3, C2) = \min(d(RO1, C2), d(RO3, C2)) = \min(515, 559) = 515$$

$$d(C3, RO6) = \min(d(RO1, RO6), d(RO3, RO6)) = \min(765, 600) = 600$$

$$d(C3, RO7) = \min(d(RO1, RO7), d(RO3, RO7)) = \min(849, 780) = 780$$

Updated table:

	<i>C3</i>	<i>C2</i>	<i>RO6</i>	<i>RO7</i>
<i>C3</i>	0	515	600	780
<i>C2</i>	515	0	559	570
<i>RO6</i>	600	559	0	702
<i>RO7</i>	780	570	702	0

6 Step 4: Merge Next Closest Offices

- **Minimum distance**: 515 (between *C3* and *C2*). - Merge into $C4 = \{RO1, RO3, RO2, RO5, RO4\}$.
 - **Update distances:**

$$d(C4, RO6) = \min(d(C3, RO6), d(C2, RO6)) = \min(600, 559) = 559$$

$$d(C4, RO7) = \min(d(C3, RO7), d(C2, RO7)) = \min(780, 570) = 570$$

Updated table:

	<i>C4</i>	<i>RO6</i>	<i>RO7</i>
<i>C4</i>	0	559	570
<i>RO6</i>	559	0	702
<i>RO7</i>	570	702	0

7 Step 5: Merge Next Closest Offices

- **Minimum distance**: 559 (between *C4* and *RO6*). - Merge into $C5 = \{RO1, RO2, RO3, RO4, RO5, RO6\}$.
 - **Updated table:**

	<i>C5</i>	<i>RO7</i>
<i>C5</i>	0	570
<i>RO7</i>	570	0

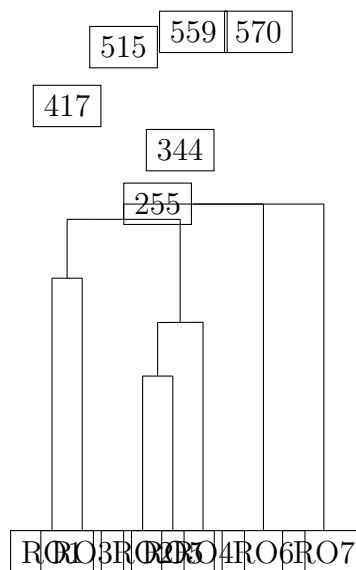
8 Step 6: Merge Final Cluster

- Merge C5 and RO7 to form the final cluster containing all offices.

9 Conclusion

We have fully clustered the seven regional offices using single-linkage, updating the distance table at every step. This forms a hierarchical clustering, which can be visualized using a dendrogram (as in the previous LaTeX example).

10 Proportional Dendrogram



11 Explanation

- Each vertical line represents an object or cluster. - Horizontal lines show merges, and their vertical position corresponds to the **distance at which clusters are merged**. - The dendrogram is **proportional**, meaning branch heights accurately reflect distances.

12 Introduction to Neural Networks

A **neural network** is a computational model inspired by the human brain, used for approximating complex functions and patterns. It consists of:

- **Input layer:** Receives the input features $X = [x_1, x_2, \dots]$.
- **Hidden layer:** Processes input using weights and activation functions.
- **Output layer:** Produces the final prediction.

- **Error function:** Measures difference between predicted output and true output y_{truth} .

13 Backpropagation Algorithm Overview

Backpropagation is used to train neural networks by updating weights to minimize error.

The main steps are:

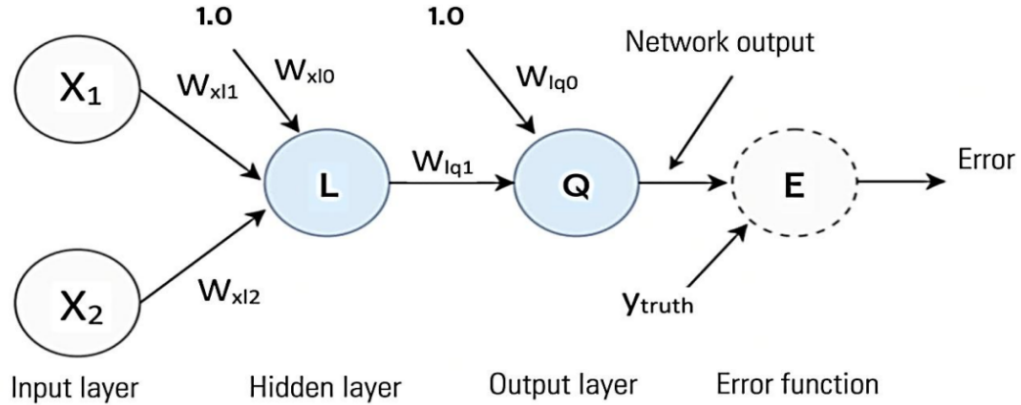
1. **Forward Pass:** Compute outputs of hidden and output layers.
2. **Compute Error:** $E = \frac{1}{2}(y_{\text{truth}} - y_{\text{pred}})^2$
3. **Backward Pass:** Compute gradients of error w.r.t weights using chain rule.
4. **Weight Update:** Adjust weights using

$$w_{ij}^{\text{new}} = w_{ij}^{\text{old}} - \eta \frac{\partial E}{\partial w_{ij}}$$

where η is the learning rate.

Q3(a) Backpropagation Algorithm Explanation

Q3 a. Briefly explain how backpropagation will be applied to the below network using pseudo code or algorithm.



Network Description

The given neural network consists of:

- **Input layer:** Two inputs X_1 and X_2
- **Hidden layer:** One neuron L
- **Output layer:** One neuron Q
- **Bias units:** Constant input 1.0
- **Error function:** E

Weights

Input to Hidden layer:

$$w_{x1}, w_{x2}, w_{x0}$$

Hidden to Output layer:

$$w_{lq1}, w_{lq0}$$

Activation Function

The sigmoid activation function is used:

$$f(z) = \frac{1}{1 + e^{-z}}$$

Derivative of sigmoid:

$$f'(z) = f(z)(1 - f(z))$$

Error Function

Mean Squared Error (MSE) is used:

$$E = \frac{1}{2}(y_{\text{truth}} - Q)^2$$

Forward Pass

Hidden neuron net input:

$$\text{net}_L = w_{x1}x_1 + w_{x2}x_2 + w_{x0}(1)$$

Hidden neuron output:

$$L = f(\text{net}_L)$$

Output neuron net input:

$$\text{net}_Q = w_{lq1}L + w_{lq0}(1)$$

Network output:

$$Q = f(\text{net}_Q)$$

Backward Pass

Output layer delta:

$$\delta_Q = \frac{\partial E}{\partial \text{net}_Q}$$

$$\delta_Q = (Q - y_{\text{truth}})f'(\text{net}_Q)$$

$$\delta_Q = (Q - y_{\text{truth}})Q(1 - Q)$$

Hidden layer delta:

$$\delta_L = \frac{\partial E}{\partial \text{net}_L}$$

$$\delta_L = \delta_Q \cdot w_{lq1} \cdot f'(\text{net}_L)$$

$$\delta_L = \delta_Q \cdot w_{lq1} \cdot L(1 - L)$$

Weight Update Rules

Using gradient descent:

$$w^{\text{new}} = w^{\text{old}} - \eta \frac{\partial E}{\partial w}$$

Hidden to Output weights:

$$w_{lq1}^{\text{new}} = w_{lq1} - \eta \cdot \delta_Q \cdot L$$

$$w_{lq0}^{\text{new}} = w_{lq0} - \eta \cdot \delta_Q \cdot 1$$

Input to Hidden weights:

$$w_{x1}^{\text{new}} = w_{x1} - \eta \cdot \delta_L \cdot x_1$$

$$w_{x2}^{\text{new}} = w_{x2} - \eta \cdot \delta_L \cdot x_2$$

$$w_{x0}^{\text{new}} = w_{x0} - \eta \cdot \delta_L \cdot 1$$

Backpropagation Algorithm (Pseudocode)

Initialize all weights

Given learning rate

FOR each training example:

```
net_L = w_x1*x1 + w_x2*x2 + w_x0
```

```
L = sigmoid(net_L)
```

```
net_Q = w_lq1*L + w_lq0
```

```
Q = sigmoid(net_Q)
```

```
E = 1/2 (y_truth - Q)^2
```

```
_Q = (Q - y_truth) * Q * (1 - Q)
_L = _Q * w_lq1 * L * (1 - L)
```

Update all weights using gradient descent

END FOR

Q3(b) Numerical Solution

Given

$$X = \begin{bmatrix} -0.7 \\ 0.2 \end{bmatrix}$$

$$w_{x1} = 0.2, w_{x2} = 0.5, w_{x0} = -0.2, w_{lq1} = -0.1, w_{lq0} = 0.4$$

$$y_{\text{truth}} = -1, \quad \eta = 0.4$$

Step 1: Forward Pass

$$\text{net}_L = (0.2)(-0.7) + (0.5)(0.2) + (-0.2)(1)$$

$$\text{net}_L = -0.24$$

$$L = \sigma(-0.24) \approx 0.440$$

$$\text{net}_Q = (-0.1)(0.440) + (0.4)(1)$$

$$\text{net}_Q = 0.356$$

$$Q = \sigma(0.356) \approx 0.588$$

Step 2: Error

$$E = \frac{1}{2}(-1 - 0.588)^2 \approx 1.26$$

Step 3: Backward Pass

$$\delta_Q = (0.588 - (-1))(0.588)(1 - 0.588)$$

$$\delta_Q \approx 0.384$$

$$\delta_L = 0.384 \times (-0.1) \times (0.440)(0.560)$$

$$\delta_L \approx -0.0095$$

Step 4: Updated Weights

$$w_{lq1}^{new} = -0.1 - 0.4(0.384)(0.440) = -0.167$$

$$w_{lq0}^{new} = 0.4 - 0.4(0.384) = 0.246$$

$$w_{x1}^{new} = 0.2 - 0.4(-0.0095)(-0.7) = 0.197$$

$$w_{x2}^{new} = 0.5 - 0.4(-0.0095)(0.2) = 0.501$$

$$w_{x0}^{new} = -0.2 - 0.4(-0.0095)(1) = -0.196$$

Final Updated Weights

$w_{x1} = 0.197$
$w_{x2} = 0.501$
$w_{x0} = -0.196$
$w_{lq1} = -0.167$
$w_{lq0} = 0.246$